GENIE FOR EXTERNAL USERS · OAUTH CUSTOM IDENTITY CLAIMS

# Solution Brief: Genie for Your Application's Users

Community reference, not official Databricks documentation. OAuth custom identity claims are announced on the Databricks *What's coming?* page; check your workspace with `sql/check_availability.sql`.

---

Your customers, members or partners can ask questions about their own data in plain language, inside your application, and get answers drawn only from what they are allowed to see. They never need a Databricks account, and your sign-in stays exactly as it is.

---

## The problem

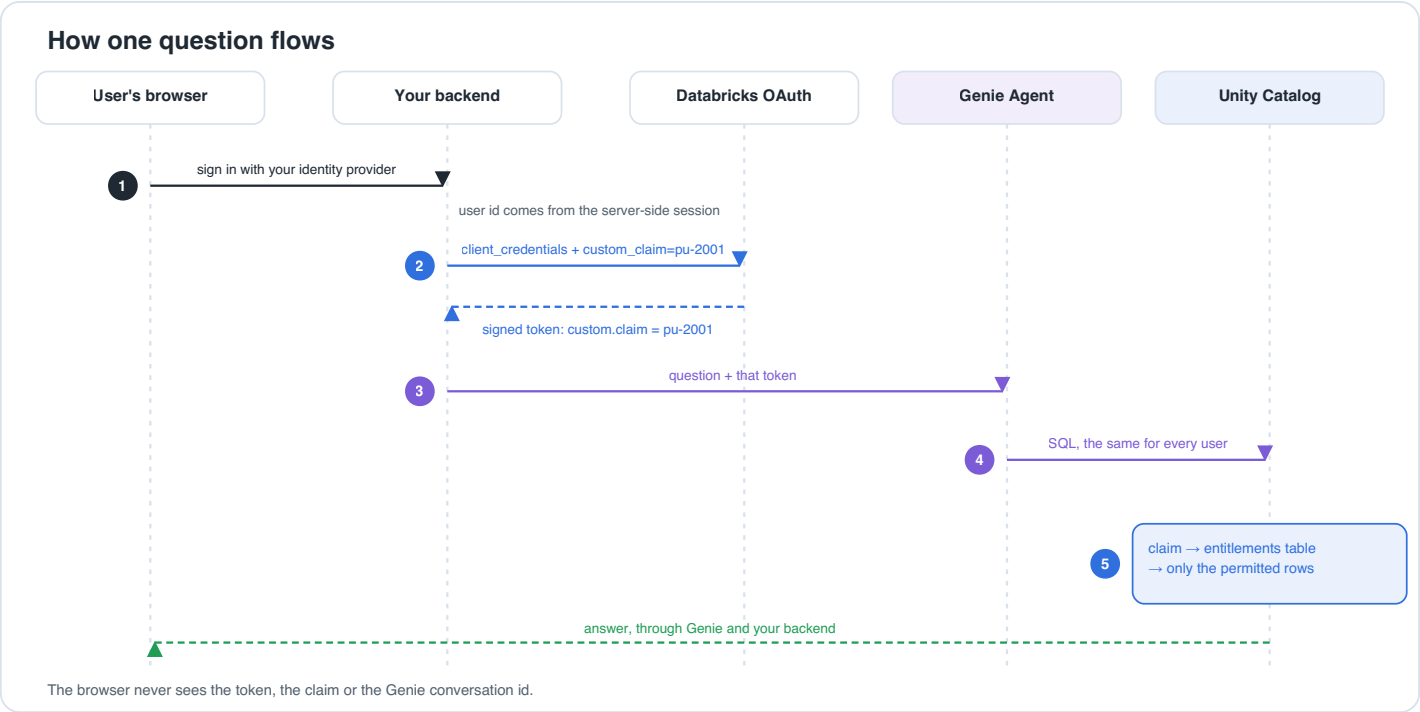
Your application has many users, often hundreds of thousands, who sign in through your own identity provider. You want each of them to ask a Genie Agent anything, and to see only their own data. Until now, every option had a catch:

| Earlier approach | The catch |
|---|---|
| Create every user in Databricks | Identity limits, and a second user lifecycle to run |
| One service principal per customer | Thousands of identities, and no room for roles inside a customer |
| Put the user's id in the prompt | An AI model can be talked into ignoring it |

---

## What changes

Your backend asks Databricks for a short-lived token for its one service principal, stamped with the signed-in user's id. Databricks signs the token, so nobody can change the id. Genie writes the SQL. Unity Catalog reads the id from the token and returns only that user's rows.




One question, end to end. The browser never sees the token.

---

# How it stays safe: two locks

Think of a coat check where one attendant serves every guest, and each guest holds a ticket stamped by the venue.

- **The first lock** is the counter: your application can reach only the views made for it, so anything nobody exposed doesn't exist for it.
- **The second lock** is the vault: every table checks the stamped ticket itself, before anything in the question is evaluated, so a clever question can't learn about anyone else's rows.

Each lock covers the other's failure mode. One misconfigured lock still leaves every user with only their own data.

---

# What we proved

Two fictional companies, a retirement plan recordkeeper and a pharmacy benefit manager, deployed on a live workspace with synthetic data:

| Check | Result |
|---|---|
| Users see exactly their own rows, roles and masks, across every view | 138 of 138 checks passed |
| Questions designed to probe other users' data through errors | 8 of 8 blocked (7 of 8 leaked without the second lock) |
| Five classic setup mistakes, made on purpose | 10 of 10 held: users still saw only their own rows |
| Cost of the second lock | about 0.15 to 0.35 seconds per question at 200 million rows |

Roles come from a simple table, so one person can be a plan sponsor and a participant, an administrator can see totals but never individuals, a consultant's access can end on a date, and a parent can see a young child's claims. A change applies on the user's next question.

---

# Who it's for

- Software companies and partners building "ask your data" into their product for many customers.
- Teams running portals for members, participants, policyholders, patients or account holders.
- Teams serving partner networks such as brokers, advisors, pharmacies or franchisees.

It works with Genie Agents through the Genie Conversation API, and with your own services through the SQL Statement Execution API and JDBC.

---

# How to start

1. **Check availability.** OAuth custom identity claims are announced on the Databricks *What's coming?* page. One SQL statement in your workspace tells you whether you have them (see the Implementation Guide).
2. **Deploy the reference.** The bundle in this repo deploys both examples, their Genie Agents and the full test suite in about an hour.
3. **Map your entitlements.** List your users' roles and scopes in the two entitlement tables, and point the views at your own tables.